EXPERIMENT 3: Apply Decision Tree and Random Forest for Classification Tasks

AIM: Apply Decision Tree and Random Forest for classification tasks

1. Dataset Source

- Dataset Name: Iris Dataset
- Source: Hugging Face
- Link: https://huggingface.co/datasets/scikit-learn/iris

2. Dataset Description

The Iris dataset is a classic dataset used for classification tasks, where the goal is to classify iris flowers into different species based on their features.

- **Number of Instances:** 150
- **Number of Features:** 4 input features + 1 target variable

Features:

- sepal_length – Length of the sepal (cm)
- sepal_width – Width of the sepal (cm)
- petal_length – Length of the petal (cm)
- petal_width – Width of the petal (cm)

Target Variable:

- species
  - 0 → Setosa
  - 1 → Versicolor
  - 2 → Virginica

Dataset Characteristics:

- Fully numerical dataset
- No missing values
- Balanced dataset
- Suitable for multi-class classification

3. Mathematical Formulation of the Algorithm

Decision Tree

Decision Trees split data based on feature values using impurity measures.

Gini Index:

$$Gini = 1 - \sum_{i=1}^{c} p_i^2$$

Where:

- (p_i) = probability of class (i)
- (c) = number of classes

Entropy:

$$Entropy = -\sum_{i=1}^{c} p_i \log_2 p_i$$

Information Gain:

$$IG = Entropy(parent) - \sum \frac{N_{child}}{N_{parent}} \cdot Entropy(child)$$

Random Forest

Random Forest is an ensemble of multiple Decision Trees.

Prediction Rule:

$$\hat{y} = \text{mode}(T_1(x), T_2(x), ..., T_n(x))$$

Where:

- (T_i) = individual decision trees

4. Algorithm Limitations

Decision Tree:

1. Prone to overfitting
2. Sensitive to small data variations
3. Can create complex trees

Random Forest:

1. Computationally expensive
2. Less interpretable than Decision Trees
3. Requires tuning of multiple parameters

5. Methodology / Workflow

Steps Involved

1. Import required libraries
2. Load Iris dataset from Hugging Face
3. Perform data preprocessing (checking for missing values, feature selection)
4. Split dataset into training and testing sets
5. Train initial Decision Tree model
6. Train initial Random Forest model
7. Define hyperparameter grid (tree depth, number of estimators, etc.)
8. Perform GridSearchCV for tuning
9. Retrieve best parameters
10. Train final optimized models
11. Evaluate model performance using accuracy, precision, recall, and F1-score
12. Generate predictions

Screenshots:-

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset
from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
dataset = load_dataset("scikit-learn/iris")

df = pd.DataFrame(dataset['train'])
df.head()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
df.info()
df.describe()
df.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   Species        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
       'Species'],
      dtype='object')
```

```
X = df.drop("Species", axis=1)
y = df["Species"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=3, random_state=42)
```

```
y_pred_dt = dt.predict(X_test)

print("=== Decision Tree ===")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))
```

```
=== Decision Tree ===
Accuracy: 1.0

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

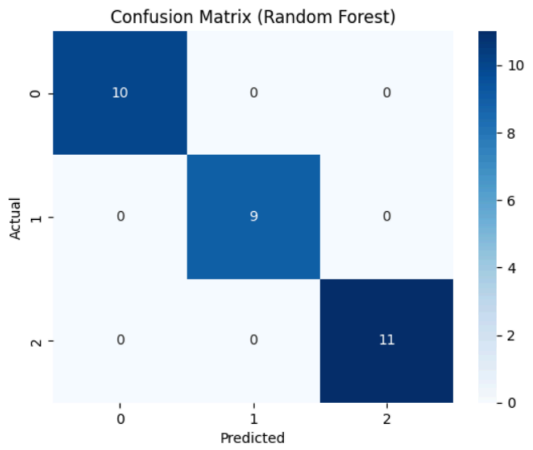
```
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

```
y_pred_rf = rf.predict(X_test)

print("=== Random Forest ===")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))
```

```
=== Random Forest ===
Accuracy: 1.0

Classification Report:
                 precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00        10
Iris-versicolor       1.00      1.00      1.00         9
 Iris-virginica       1.00      1.00      1.00        11

       accuracy                           1.00        30
      macro avg       1.00      1.00      1.00        30
   weighted avg       1.00      1.00      1.00        30
```

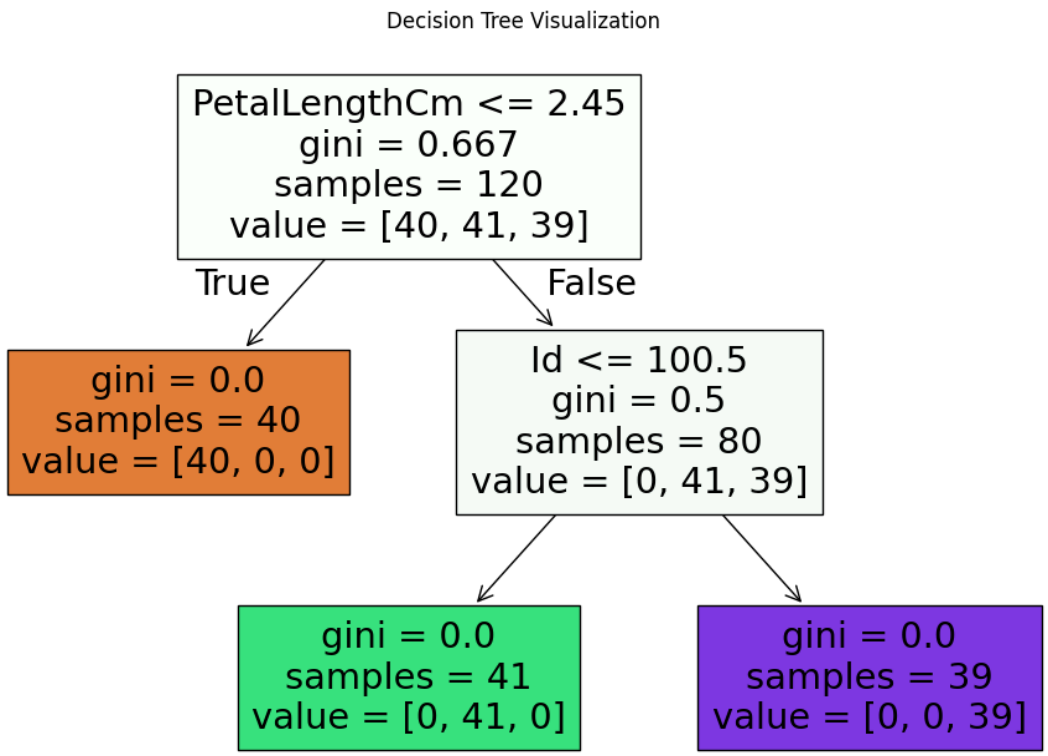
```
cm = confusion_matrix(y_test, y_pred_rf)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title("Confusion Matrix (Random Forest)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
results = pd.DataFrame({
    "Model": ["Decision Tree", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_dt),
        accuracy_score(y_test, y_pred_rf)
    ]
})

results
```

	Model	Accuracy
0	Decision Tree	1.0
1	Random Forest	1.0

```
from sklearn.tree import plot_tree

plt.figure(figsize=(12,8))
plot_tree(dt, filled=True, feature_names=X.columns)
plt.title("Decision Tree Visualization")
plt.show()
```



6. Performance Analysis

Evaluation Metrics Used:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Interpretation:

- **Accuracy:** Overall correctness of classification
- **Precision:** Correctness of predicted class labels
- **Recall:** Ability to identify actual class instances
- **F1-score:** Balance between precision and recall

Observations:

- Decision Tree performs well but may overfit
- Random Forest provides better generalization
- Ensemble learning improves accuracy and stability
- Iris dataset gives high accuracy due to clear class separation

7. Hyperparameter Tuning

Parameters Tuned:

Decision Tree:

- max_depth
- criterion (gini, entropy)

Random Forest:

- n_estimators
- max_depth
- max_features

Method Used:

- GridSearchCV

Process:

- Tested multiple parameter combinations
- Used cross-validation for evaluation
- Selected best parameters based on accuracy

Best Parameters:

- Optimal tree depth and splitting criteria
- Optimal number of trees in Random Forest

Impact on Performance:

- Reduced overfitting in Decision Tree
- Improved generalization in Random Forest
- Increased model accuracy

8. Conclusion

In this experiment, Decision Tree and Random Forest algorithms were applied to the Iris dataset for classification. Both models performed well, with Random Forest achieving better accuracy and generalization. Hyperparameter tuning further improved performance. The results show that ensemble methods like Random Forest are more robust compared to single models.